JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Interface Future<V>

Type Parameters:

`V - The result type returned by this Future's get method`

All Known Subinterfaces:

`Response<T>, RunnableFuture<V>, RunnableScheduledFuture<V>, ScheduledFuture<V>`

All Known Implementing Classes:

`CompletableFuture, CountedCompleter, ForkJoinTask, FutureTask, RecursiveAction, RecursiveTask, SwingWorker`

`public interface` **`Future<V>`**

A `Future` represents the result of an asynchronous computation. Methods are provided to check if the computation is complete, to wait for its completion, and to retrieve the result of the computation. The result can only be retrieved using method `get` when the computation has completed, blocking if necessary until it is ready. Cancellation is performed by the `cancel` method. Additional methods are provided to determine if the task completed normally or was cancelled. Once a computation has completed, the computation cannot be cancelled. If you would like to use a `Future` for the sake of cancellability but not provide a usable result, you can declare types of the form `Future<?>` and return `null` as a result of the underlying task.

Sample Usage (Note that the following classes are all made-up.)

```
interface ArchiveSearcher { String search(String target); }
class App {
  ExecutorService executor = ...
  ArchiveSearcher searcher = ...
  void showSearch(final String target)
```

```
      throws InterruptedException {
    Future<String> future
      = executor.submit(new Callable<String>() {
        public String call() {
            return searcher.search(target);
        }});
    displayOtherThings(); // do other things while searching
    try {
      displayText(future.get()); // use future
    } catch (ExecutionException ex) { cleanup(); return; }
  }
}
```

The `FutureTask` class is an implementation of `Future` that implements `Runnable`, and so may be executed by an `Executor`.
For example, the above construction with `submit` could be replaced by:

```
FutureTask<String> future =
  new FutureTask<String>(new Callable<String>() {
    public String call() {
      return searcher.search(target);
  }});
executor.execute(future);
```

Memory consistency effects: Actions taken by the asynchronous computation *happen-before* actions following the corresponding `Future.get()` in another thread.

Since:

`1.5`

See Also:

`FutureTask`, `Executor`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
boolean	**cancel**(boolean mayInterruptIfRunning)	Attempts to cancel execution of this task.
V	**get**()	Waits if necessary for the computation to complete,

		and then retrieves its result.
V	**get**(long timeout, **TimeUnit** unit)	Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.
boolean	**isCancelled**()	Returns `true` if this task was cancelled before it completed normally.
boolean	**isDone**()	Returns `true` if this task completed.

Method Detail

cancel

`boolean cancel(boolean mayInterruptIfRunning)`

Attempts to cancel execution of this task. This attempt will fail if the task has already completed, has already been cancelled, or could not be cancelled for some other reason. If successful, and this task has not started when `cancel` is called, this task should never run. If the task has already started, then the `mayInterruptIfRunning` parameter determines whether the thread executing this task should be interrupted in an attempt to stop the task.

After this method returns, subsequent calls to `isDone()` will always return `true`. Subsequent calls to `isCancelled()` will always return `true` if this method returned `true`.

Parameters:

mayInterruptIfRunning - true if the thread executing this task should be interrupted; otherwise, in-progress tasks are allowed to complete

Returns:

false if the task could not be cancelled, typically because it has already completed normally; true otherwise

isCancelled

```
boolean isCancelled()
```

Returns `true` if this task was cancelled before it completed normally.

Returns:

true if this task was cancelled before it completed

isDone

```
boolean isDone()
```

Returns `true` if this task completed. Completion may be due to normal termination, an exception, or cancellation -- in all of these cases, this method will return `true`.

Returns:

true if this task completed

get

```
V get()
throws InterruptedException,
       ExecutionException
```

Waits if necessary for the computation to complete, and then retrieves its result.

Returns:

the computed result

Throws:

CancellationException - if the computation was cancelled

ExecutionException - if the computation threw an exception

InterruptedException - if the current thread was interrupted while waiting

get

```
V get(long timeout,
      TimeUnit unit)
throws InterruptedException,
      ExecutionException,
      TimeoutException
```

Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.

Parameters:

`timeout - the maximum time to wait`

`unit - the time unit of the timeout argument`

Returns:

`the computed result`

Throws:

`CancellationException - if the computation was cancelled`

`ExecutionException - if the computation threw an exception`

`InterruptedException - if the current thread was interrupted while waiting`

`TimeoutException - if the wait timed out`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.